MinAI: An Introduction to Language Models

A companion textbook for `minai.cpp`.

Preface

This book is for someone who has taken an algebra class, and probably a calculus class, and maybe a linear-algebra class, and now wants to understand how ChatGPT actually works — not as a metaphor, not as a vibe, but as a program that you could type out yourself. If you can read basic C or C++, the program at your side (`minai.cpp`) is the smallest possible version of that program. It is about 540 lines. It learns to reverse a short list of digits. And every single mechanism it uses is a mechanism inside every large language model on earth. When you finish this book, the only difference between what you understand and what GPT-4 does will be *quantity*: more layers, more heads, more parameters, more training data. Not one new idea.

Who this is for

I wrote this for a particular kind of reader — someone who took calculus at some point but was never told *what calculus is for*. You may have computed some derivatives, solved some integrals, and then forgotten the whole thing because nothing you do day-to-day involves infinitesimals. That is fine. Across this book I assume:

- You are comfortable with functions: `y = f(x)`, `y = 3x + 2`, that kind of thing.
- You have seen derivatives (slope of a curve). You may not remember how to compute one by hand; I will re-derive the few we need.
- You have seen vectors — an arrow, or equivalently an ordered list of numbers like `(1, 4, -2)`.
- You have seen matrices, or you will trust me for a paragraph while I remind you.
- You can read code; you do not need to know C++ specifically.

Every concept beyond this is built from scratch.

How to use this book

`minai.cpp` is a complete, self-contained C++ transformer. It lives in the same directory as this file. Build it and run it first:

```
make
./minai
```

It prints a training log and a demo. That is the thing this book explains. Every chapter points to specific parts of `minai.cpp`. Read the code alongside — not to memorize, but to see each concept *running as code* the moment you read about it.

Roadmap

- Chapters 1–3 set up the problem: what a model is, what "learning" means, and the minimum linear-algebra vocabulary we need.
- Chapters 4–7 describe the **forward pass** of a transformer: text → numbers → attention → probabilities.
- Chapters 8–10 describe **training**: the loss function, the calculus (finally), the weight-update loop.
- Chapters 11–12 cross the bridge from math to hardware: how abstract calculus becomes matrix multiplication, and why GPUs and their memory have become the limiting resource of the entire AI industry right now.
- Chapter 13 is the view from the top: here is what GPT-4 adds on top of what you just read.
- Appendix A is a side story about doing all of this in integer arithmetic on a 1970s minicomputer.
- Appendix B is a set of exercises.

There is no math in this book you could not do on paper, given a quiet afternoon. There is nothing mysterious about what the code is doing. I promise.

Chapter 1 — What a model is

Forget machine learning for a moment. You have been working with "models" since middle school. Here is one:

```
y = 3x + 2
```

Given an input `x`, it produces an output `y`. The numbers `3` and `2` are **parameters** — they are what make this model *this* model rather than some other line like `y = 5x − 1`. Change the parameters, and you have a different function.

This is a tiny model. Two parameters. It can represent any line; nothing more.

What "learning" means

Suppose I give you the following data:

```
x = 1, y = 5
x = 2, y = 8
x = 3, y = 11
```

You spot immediately that `y = 3x + 2` fits. You just *learned* that model from data. To a human, the process is visual: plot the points, eyeball the line, write down the equation. To a computer, the process has to be mechanical:

1. **Guess** parameter values, maybe randomly.
2. **Measure** how badly your current guess matches the data.
3. **Nudge** the parameters in a direction that reduces the badness.
4. Repeat steps 2–3 until the badness is small.

That four-step recipe is the *entire* theory of how neural networks learn. It is called **gradient descent**, and once you see it, you will see it everywhere.

Step 3 is the hard part: "nudge them in a direction that reduces the badness." What does *direction* mean for numbers? How do you pick it? You need to know the **slope** of the badness function at your current point — because slope tells you which way is up (and therefore which way is down). Slopes are what derivatives give you. That is what calculus is for, in one sentence. We will spell it out in Chapter 2.

Why a machine needs this machinery

For `y = 3x + 2`, you could just solve by hand. Why build the four-step machinery? Because it *keeps working* when the model has a million parameters instead of two, and when there is no clean formula waiting to be guessed. The function that maps "picture of a cat" → "the label 'cat'" has no closed-form expression. The function that maps "first half of a sentence" → "next word" has no closed-form expression either. But both functions can be **approximated** by setting up a model with millions of parameters and running gradient descent until the badness is low.

`minai.cpp` has 2,240 parameters by default. It maps an 8-digit input to an 8-digit output. A closed-form rule exists ("reverse the list"), but the model doesn't know that. Training has to find, from scratch, parameter values that encode that rule. If you can accept that training a 2,240-parameter model to reverse digits works by the *same four-step recipe* as training a 175-billion-parameter model to write sonnets, you're well on your way.

> **Read along:** Open `minai.cpp` and look at **Part 4 (Model Parameters)**. Count the float arrays declared as globals: `token_emb`, `pos_emb`, `Wq`, `Wk`, `Wv`, `W1`, `W2`, `Wout`. Each entry in each array is one of the parameters gradient descent will adjust. The total matches the "Parameters: 2240" number the program prints at startup.

Chapter 2 — Why calculus, finally

Imagine you are in a hilly landscape, in thick fog, and you want to reach the lowest point. You cannot see far enough to know where the valley is. What *can* you do? You can feel the slope of the ground right under your feet. If the ground slopes down to the north, you step north. A few feet later you check again: maybe now it slopes down to the northeast. You adjust and step again. Repeat. If the landscape is reasonably well-behaved, you will eventually reach a low point — not necessarily the global minimum of the map, but *a* local minimum good enough for most purposes.

That is gradient descent in one paragraph. Everything else is details.

The "badness function" has a shape

Back to our line-fitting problem: given data `(x_i, y_i)` for `i = 1..N`, and a model `f(x) = mx + b` with parameters `(m, b)`, define the **loss** (the technical word for badness) as the average squared error:

```
L(m, b) = (1/N) * Σ_i ( y_i - (m*x_i + b) )^2
```

This is a function of `(m, b)`. Pick any two numbers, plug them in, get a single number out (the loss). The space of possible `(m, b)` values is just a 2D plane, and `L(m, b)` is the height of the "landscape" above each point. Our goal is to find the `(m, b)` at the bottom of that landscape.

Now: what does a derivative *tell you*?

For a function of one variable `f(x)`, the derivative `f'(x)` at a point tells you how much `f` changes per small change in `x`. If `f'(5) = 3`, it means: right around x=5, moving x up by a tiny bit Δ makes f go up by about 3Δ. If `f'(5) = -7`, moving x up makes f go down (by about 7Δ). In both cases, the derivative is the *slope* at that point.

For a function of **two** variables `L(m, b)`, the generalization is a **gradient** — a little arrow with two components, one for each variable:

```
∇L = ( ∂L/∂m , ∂L/∂b )
```

That notation looks scary. It just means: the first number is "how much does L change per unit change in m, holding b fixed", and the second is the mirror image for b. Together, the two numbers point in the direction where L goes *up* the fastest. Flip the sign and you get the direction where L goes *down* the fastest. That is the direction to step.

The update rule

Given a small positive number `α` (the **learning rate**, a knob you pick):

```
 m := m - α * ∂L/∂m
b := b - α * ∂L/∂b
```

That is it. That is gradient descent. Compute the gradient, take a small step opposite to it, repeat. After thousands of iterations, `(m, b)` will settle near the minimum of `L`, which is exactly the parameter values that best fit the data. The calculus course you took gives you the tool (derivatives) to compute that direction.

Now plug in a real number of parameters: 2,240 for MinAI, 175 billion for GPT-3. Same procedure. The gradient has 2,240 components; you step opposite to it. Nothing conceptually harder. The only new problem is *how to compute the gradient efficiently* when the function is a huge composition of matrix multiplies — and that is what Chapter 9 is about.

> **Read along:** In `minai.cpp`, **Part 11 (Optimizer)** is three loops that implement exactly this update rule: `parameter -= learning_rate * gradient_of_parameter`. Three lines of "machine-learning theory" in the middle of each loop. That is the whole optimizer.

Chapter 3 — Vectors and matrices, the bare minimum

Models with billions of parameters need more structured representations than single numbers. You have to upgrade from scalars (single numbers) to **vectors** (ordered lists of numbers).

Vectors

A vector is an ordered list of numbers:

```
v = (0.3, -1.2, 0.8, 0.05)
```

This particular vector has **dimension** 4 because it has four components. In MinAI, `D_MODEL = 16`, so every position in the sequence is internally represented by a 16-dimensional vector.

You can add two vectors of the same dimension: add them component by component. You can multiply a vector by a scalar: multiply each component. The operation we care about most is the **dot product** of two vectors of the same dimension:

```
 a = (a_0, a_1, a_2, a_3)
b = (b_0, b_1, b_2, b_3)
a · b = a_0*b_0 + a_1*b_1 + a_2*b_2 + a_3*b_3
```

Multiply componentwise, sum the products. One single number comes out.

The dot product has a geometric meaning: it measures how *aligned* two vectors are. If a and b point in the same direction, the dot product is large and positive. If they are perpendicular, it's zero. If they point opposite,

it's negative. Formally: `a · b = |a| |b| cos(θ)`, where `θ` is the angle between them.

Hold onto that intuition. In Chapter 5 you will see the machine compute one of these for every pair of tokens in the sequence, and interpret it as "how well does this position's query match that position's advertised key". Dot product = similarity is the whole mental model.

Matrices

A matrix is a rectangular grid of numbers. A matrix `W` with `m` rows and `n` columns can be thought of either as "a collection of `m` vectors of dimension `n`" or "a collection of `n` vectors of dimension `m`". Same numbers, two readings.

The operation we need is **matrix–vector multiplication**. If `W` is `m × n` and `v` is a vector of dimension `n`, then `W * v` is a vector of dimension `m`, and its `i`-th component is the dot product of row `i` of `W` with `v`:

```
(W * v)_i = Σ_j  W[i][j] * v[j]
```

Matrix–matrix multiplication is the same thing repeated: for each column of the second matrix, multiply it by the first. The product has shape `(rows-of-first) × (columns-of-second)`.

Think of a matrix multiplication `Wv` as: "I am running `v` through a bunch of dot products (one per row of `W`), and producing a new vector where each component is one of those similarity scores." That's the mental picture. A neural-network "linear layer" is exactly one of these multiplications (plus sometimes a bias added at the end, though MinAI skips biases for simplicity).

> **Read along:** `minai.cpp` Part 8, **step 2** of `forward_block`. The nested loops compute `Q = X @ Wq`, `K = X @ Wk`, `V = X @ Wv`. Three matrix multiplies. Each innermost `for (int i = 0; i < D_MODEL; ++i)` loop is one dot product. Look at them and trust that you understand the bones of the arithmetic.

Chapter 4 — Tokens, embeddings, and position

A language model takes in symbols (words, digits, characters) and produces more symbols. But a neural network does math on vectors, not symbols. How do we bridge that gap?

The vocabulary

First, we decide on a finite alphabet of symbols the model knows about. In MinAI, the alphabet is just the ten digits `0..9`. We call this set the **vocabulary**, size `VOCAB = 10`. In GPT-3, the vocabulary is about 50,000

“subword” pieces (not whole words, and not letters — chunks like `"berg"` or `" the"`). The tokenization story is a whole other book; for us, one digit is one token.

Each symbol is assigned an integer **token id** — digit 0 gets id 0, digit 5 gets id 5, etc. GPT-3’s tokenizer has a more complex assignment but does the same job.

Embeddings: turn token ids into vectors

Now for the bridge. We keep a lookup table `token_emb` of shape `[VOCAB][D_MODEL]`. That is, 10 rows (one per token) and 16 columns (the vector dimension). The `t`-th row of this table is the vector the model will use whenever it encounters token `t`. This table is **learned** — it starts out random and gradient descent adjusts it over training until each row becomes a useful encoding of the corresponding token.

That is worth pausing on. *The model decides for itself how to represent the digit “3”*, by picking a 16-dimensional vector. It is not given one. It tries random ones and keeps the ones that make its predictions better. In a trained GPT-3, the embedding vectors of words with similar meaning end up close to each other in vector space — an emergent consequence of training, not a rule anyone wrote down.

Position embeddings

There is a subtlety. Look ahead to Chapter 5: attention is a sum over positions weighted by similarity. Swap the order of two input tokens, and the sum is unchanged. A pure attention model cannot tell “the cat sat on the mat” from “on the mat the cat sat.” That is a problem.

Solution: keep a second lookup table `pos_emb` of shape `[SEQ_LEN][D_MODEL]`. Row `t` is a vector unique to position `t` in the input. *Add* the position vector to the token vector:

```
X[t] = token_emb[tokens[t]] + pos_emb[t]
```

Now position 0 and position 5 look different to the network even if they contain the same token. Symmetry broken.

> **Read along:** `minai.cpp` Part 8, **step 1** of `forward`. This is the literal line: `X_block[0][t][d] = token_emb[tokens[t]][d] + pos_emb[t][d];`. This is the *only place* in the whole program where raw integer token ids enter the network. Past that one line, it is all real-valued vectors.

Chapter 5 — Attention, the actual new idea

Attention is what put the “T” in GPT. Everything else in a transformer — embeddings, residuals, feed-forward networks — had existed for years before transformers. Attention is the new thing. Once you get it, the rest of the architecture falls into place.

The problem it solves

In a sentence, the meaning of a word often depends on *some* other word, not every other word. “The trophy didn’t fit in the suitcase because **it** was too big.” Does *it* refer to the trophy or the suitcase? To answer, you need to look at some specific other word (size-related context). A fixed recipe — “always look at the previous word” or “average everything” — won’t work, because *which* other word matters depends on the meaning being computed.

Attention is a mechanism that lets each position *dynamically* decide which other positions are relevant, and pull information from them.

The soft-dictionary analogy

Imagine a dictionary lookup. You provide a key, the dictionary finds the matching entry, returns the value. That is **hard** lookup: you either match or you don’t.

Attention is **soft** lookup:

1. Every position `u` in the sequence emits a **key** vector `K[u]` advertising what it has, and a **value** vector `V[u]` which is the content it would contribute if chosen.
2. When I am at position `t`, I emit a **query** vector `Q[t]` describing what I am looking for.
3. For each position `u`, I compute the **similarity** between my query and its key — a single number, the dot product `Q[t] · K[u]`.
4. I pass all those similarity scores through a softmax (Chapter 6) which turns them into a probability distribution: now they are positive and sum to 1. Call them `attn[t][u]`.
5. My new representation is the attention-weighted sum of values: `Σ_u attn[t][u] · V[u]`.

If the similarities strongly favor one position `u*`, the softmax produces something close to one-hot: 1 at `u*`, near-zero elsewhere. The weighted sum then just copies `V[u*]`. That is “hard” dictionary lookup as a limit case. With softer similarities you get a blend of multiple values. The word “soft” in “soft dictionary” means *the choice is differentiable*: you can compute how much each `Q`, `K`, and `V` should change to reduce the loss. Unlike a hard lookup, which has no derivative, this mechanism is trainable.

Where do Q, K, V come from?

From the input vector X, by three learned linear layers:

```
 Q[t] = X[t] @ Wq
K[t] = X[t] @ Wk
V[t] = X[t] @ Wv
```

Each of those is a matrix multiplication: `Wq`, `Wk`, `Wv` are `D_MODEL × D_MODEL` matrices whose entries are learned parameters. So the network learns *three different views* of each input vector, one for use as a query, one as a key, one as a value. What makes a good key-view vs a good query-view? Whatever makes the final predictions better. Gradient descent sorts it out.

The scaled part

One wrinkle. When we compute `Q · K`, the numbers grow with the dimension `D_MODEL`, because we are summing `D_MODEL` products. With `D = 16` they are modest; with `D = 12,288` they are huge. Huge scores send softmax into its saturated regime (one score wins by a landslide) and gradients vanish. The fix: divide by `sqrt(D_MODEL)` before the softmax. That keeps the variance of the scores roughly 1, regardless of dimension.

This is why transformer attention is officially called "scaled dot-product attention". Without the scale, it doesn't train at large D. With it, it does.

The matrix form

We can write the whole thing as three matrix ops on the whole batch at once:

```
scores = Q @ K^T / sqrt(D_MODEL)      # shape [SEQ_LEN, SEQ_LEN]
attn   = softmax(scores, axis=-1)     # same shape, each row sums to 1
attn_o = attn @ V                     # shape [SEQ_LEN, D_MODEL]
```

That is self-attention. `attn_o[t]` is the attention-weighted sum of values for position `t`. Do this for all `t` in parallel and you've just done one "attention layer".

> **Read along:** `minai.cpp` Part 8, **steps 3, 4, 5** of `forward_block`. The three nested-loop blocks correspond exactly to the three lines above. Look at the attention matrix the demo prints after training (in `--blocks=2` especially): rows 6 and 7 put most of their weight on column 0, which is exactly "pay attention to the right positions in order to reverse".

Chapter 6 — Softmax

After Chapter 5 we have a grid of "similarity scores" that are arbitrary real numbers — some positive, some negative. We need probabilities (positive, summing to 1) so we can interpret them as a weighting. That is what softmax is for.

The formula

```
softmax(x)_i = exp(x_i) / Σ_j exp(x_j)
```

Three things to notice:

1. `exp(x)` is always positive. Negative inputs produce small positive outputs; positive inputs produce large positive outputs. All signs gone.
2. Dividing by the sum forces the outputs to sum to 1.
3. Exponentials *amplify* differences. If one input is meaningfully larger than the rest, it gets most of the probability. Softmax is therefore "soft argmax" — it picks the biggest input, but softly rather than making a hard choice.

Why `exp`, not some other way to make numbers positive?

You could square. You could take absolute value. Why the exponential? Two reasons.

First, `exp` has a beautiful property with calculus: `d/dx exp(x) = exp(x)`. The derivative is itself. This makes gradients of softmax clean and cheap. In Chapter 9 you will see that the combination *softmax + cross-entropy* has a ridiculously simple derivative, and the reason is `exp` playing nicely with `log`.

Second, in the information-theoretic setup where loss is `-log(probability)`, softmax is the probability distribution that maximizes entropy for a given set of expected-value constraints. That is a deep reason but it is beyond this chapter.

The "subtract the max" trick

The formula as written has a numerical problem. `exp(100)` is about `10^43`, which does not fit in a float. If any score is that large, the whole computation overflows. But notice:

```
softmax(x + c) = softmax(x)   for any constant c
```

Why? Because `exp(x_i + c) = exp(c) exp(x_i)`, and the `exp(c)` factor appears in both numerator and denominator and cancels. So we can subtract any constant from all inputs before computing softmax, with no change to the output. In practice: subtract the max input. Now the largest value is 0, `exp(0) = 1`, and nothing overflows.

> **Read along:** `minai.cpp` Part 7, `softmax_inplace`. Three loops: find max, subtract and exponentiate, divide by sum. The subtract-max is in the second loop (`row[i] = exp(row[i] - max_val)`). This trick is not an optimization — it is correctness. Without it the program would produce `inf` on large inputs.

And see **Part 15** for the bonus: the PDP-11 had no `exp()` function at all. The original program used a 256-entry lookup table indexed by `(max - x[i]) >> 5` to produce `exp(-k/8)` in fixed-point. This is the same trick modern hardware uses to quantize LLMs for edge deployment. Appendix A has more.

Chapter 7 — Putting the block together

We now have the tools for one full forward pass of a single transformer block. Here is the whole recipe in order:

```
X                                          # input: [T, D]
│
├─→ Q = X @ Wq
├─→ K = X @ Wk                            # attention
├─→ V = X @ Wv                            # sub-layer
│   scores = Q @ K^T / sqrt(D)            #
│   attn   = softmax(scores)              #
│   attn_o = attn @ V                     #
│
H1 = X + attn_o                            # residual
│
├─→ ff_pre = H1 @ W1                      # FFN
│   ff_act = relu(ff_pre)                 # sub-layer
│   ff_out = ff_act @ W2                  #
│
H2 = H1 + ff_out                           # residual
```

There are three new things in this picture besides what we've already covered.

Residual connections

Notice `H1 = X + attn_o`. The attention block does **not** output a replacement for X; it outputs a *delta* that gets added to X. Why?

Consider training a 96-layer network. Early in training, the weights are random, so each layer's output is essentially noise. If each layer *replaces* its input with its output, the signal has to survive 96 rounds of random transformations, which rapidly washes it out — and no useful gradient reaches the early layers. Contrast this with the residual formulation: if each layer is initialized to near-zero, then `H1 ≈ X`, and the network starts life as (approximately) the identity function. Training then gradually teaches each layer to *modify* its input rather than replace it. The signal path from input to output always exists as a clean sum.

Residual connections are the single most important architectural invention for training deep networks. Introduced in 2015 for image classification ("ResNet"), adopted by transformers from day one.

The feed-forward sub-layer (FFN)

Attention moves information *across* positions. But some transformations happen within a single token — "if this position's vector looks like X, transform it to Y". That is what the FFN does. It is a two-layer MLP applied to each position independently:

```
ff_pre = H1 @ W1                    # expand:  D_MODEL → D_FF
ff_act = relu(ff_pre)               # nonlinearity
ff_out = ff_act @ W2                # project: D_FF → D_MODEL
```

`relu(x) = max(0, x)`. A pointwise nonlinearity. Without a nonlinearity between the two matrix multiplies, `W1` and `W2` could be combined into a single matrix (linear * linear = linear), and the FFN would have no more expressive power than a single linear layer. The ReLU breaks linearity and lets the FFN represent more complex per-token transformations.

FFN is where **most of the parameters** in a large transformer live. In GPT-3, roughly 2/3 of the parameter count is in FFN matrices. Attention gets the headlines; FFN does the bulk of the storage.

Stacking blocks

One block of (attention + FFN) is one **layer** of a transformer. Stacking `N` layers means: feed the output of block `b` into block `b+1`. The first block sees the input embedding; the last block's output goes to the final output projection. In `minai.cpp` you toggle the stack with `--blocks=N`.

Each block has its *own* copies of `Wq, Wk, Wv, W1, W2`; they are independent and learned separately. More blocks = more capacity = can represent more complicated functions, at the cost of more parameters and more training.

The output projection

After the last block, every position still has a vector of dimension `D_MODEL = 16`. We need a vector of size `VOCAB = 10` so we can ask "which digit is this?". One more matrix multiplication:

```
logits = X_final @ Wout       # [T, D_MODEL] @ [D_MODEL, VOCAB] = [T, VOCAB]
```

The output of this multiplication is called a **logit** vector (one per position in the sequence). A logit is just a raw, un-normalized score — it can be any real number, positive or negative. The word comes from "log-odds" in classical statistics, but in modern neural-network usage it has drifted to mean simply *the number you feed into softmax*. If a logit is large and positive, the corresponding digit is strongly favored; large and negative, strongly disfavored; zero, neutral. You will see the word "logit" in every paper and every library's API; it is worth memorizing.

Softmax over the `VOCAB` axis then turns each row of logits into a probability distribution over the 10 digits. Argmax of that distribution is the predicted digit.

> **Read along:** `minai.cpp` Part 8, all of `forward_block` and `forward`. You now have the vocabulary to read every line of both functions.

LayerNorm: the piece that makes deep stacks trainable

There is one more ingredient to a modern transformer block, and it is the only one that matters for training very deep networks: **layer normalization** (LayerNorm for short). It is optional in MinAI (`--layernorm=1`) but always on in real LLMs.

The problem it solves. The residual connections from earlier in this chapter help a lot, but they are not enough once you stack tens of layers. The scale of the activation vectors at each layer tends to drift — either shrinking toward zero (vanishing gradients, nothing trains) or blowing up toward infinity (`NaN` , everything dies). In MinAI you can see this happen on your own machine. Run:

```
./minai --blocks=32 --layernorm=0 --steps=200
```

Within about 150 steps the loss becomes `NaN` and accuracy sticks at 1/8. The 32-layer model's gradients exploded. Now run:

```
./minai --blocks=32 --layernorm=1 --steps=1000
```

Same architecture, same depth, same training data. LayerNorm added. Loss drops smoothly; accuracy hits 8/8 well before 1000 steps. That is the lesson in a nutshell.

What LayerNorm does. For each token's D_MODEL-wide vector, compute its mean and standard deviation *across its own components* (not across the batch, not across the sequence — just across the 16 numbers in that one vector). Subtract the mean and divide by the std so the vector has zero mean and unit variance. Then multiply and shift by two learned per-channel vectors, called **gain** and **bias**:

```
 mean  = average of x[0..D-1]
std   = std-dev of x[0..D-1]
xhat  = (x - mean) / std              # normalized: zero mean, unit variance
y     = xhat * gain + bias            # learned per-channel rescale/shift
```

The subtract-and-divide fixes the scale. The gain and bias give the model the freedom to un-do that normalization if it wants to — they start at 1 and 0 respectively, so LayerNorm is initially the identity, and training decides whether and how much to stray from it.

Where we put it. There are two conventions. In the original 2017 paper, LayerNorm came *after* each sub-layer's output (the "Post-LN" style). Modern transformers (GPT-2, GPT-3, GPT-4) place it *before* each sub-layer's input (the "Pre-LN" style):

```
 H1 = X  + Attn(LN(X))       # Pre-LN: LN before attention, residual wraps X
H2 = H1 + FFN(LN(H1))       # Pre-LN: LN before FFN, residual wraps H1
```

MinAI uses Pre-LN — it is significantly more stable for deep networks. One more LayerNorm sits just before the final output projection so the logits come out at a predictable scale.

The parameter cost is trivial. Two D_MODEL vectors per LN × 2 LNs per block + 1 final LN = 4 × D_MODEL + 2 × D_MODEL = 6 × D_MODEL per block ≈ 96 extra parameters per block in MinAI. Negligible.

Why it works (the intuition, not the proof). Normalizing the inputs to each sub-layer keeps every sub-layer operating on vectors of similar magnitude throughout training. The gradients computed backward through each sub-layer therefore also stay at similar magnitudes. Vanishing and exploding gradients, both of which are caused by scale drift compounding across many layers, are prevented by design. The formal math is a short derivation you will meet in Chapter 9 if you want it.

The takeaway. Once you add LayerNorm, the number of layers you can stack stops being limited by numerical stability and starts being limited by your patience and compute budget. GPT-3's 96 layers are only possible because every layer is sandwiched between LayerNorms keeping the scales tame. You can reproduce that exact depth in MinAI with:

```
./minai --blocks=96 --layernorm=1 --steps=2000
```

It will run slowly because 96 blocks × 8 positions × a lot of small matrix multiplies, but it will actually train — same code path, same math, same mechanism as GPT-3. Ninety-six times. On the fixed-input task 2000 steps is plenty to hit 8/8 and plateau.

> **Read along:** `minai.cpp` Part 7b. Two functions, `layernorm_forward_row` and `layernorm_backward_row`, each about ten lines. The forward is the formula above. The backward is the derivative of the formula, derived exactly as Chapter 9 will teach you to derive things: chain rule applied one step at a time.

Chapter 8 — Loss: measuring how wrong

We can now run the model forward and get probabilities. To train the model, we need to measure how wrong those probabilities are given the correct answer, and use that measurement to drive gradient descent.

What “being wrong” feels like in probability

Suppose the correct answer is digit 7, and the model puts probability 0.8 on it. Not bad. Put 0.01 on it and you should be more embarrassed. Put 0.00001 and you should be *very* embarrassed — you have not just got the wrong answer, you were confidently wrong. A good loss function should punish confident wrongness harder than tentative wrongness.

Cross-entropy does this perfectly:

```
L = -log(p_correct)
```

where `p_correct` is the probability the model assigned to the correct answer. Run the numbers:

- `p = 1.0` → `L = 0`. Perfect.
- `p = 0.5` → `L ≈ 0.69`. Mildly off.
- `p = 0.1` → `L ≈ 2.3`. Quite wrong.
- `p = 0.001` → `L ≈ 6.9`. Very wrong.
- `p = 0` → `L = ∞`. Infinite loss for total confidence in a wrong answer.

Notice the hyperbolic curve. The `-log` penalty goes to infinity as the predicted probability approaches 0. That is the shape we want: the model is essentially *forced* to never be confidently wrong.

Why cross-entropy instead of squared error?

You could, in principle, use the squared error between the predicted distribution and a one-hot target `(0, 0, …, 1, …, 0)`. It is pedagogically tempting. But it has a specific gradient problem: when the model is *very* wrong, the squared-error gradient is *small*, because you are already far into the flat tail of a quadratic. Cross-entropy does the opposite — the wronger you are, the bigger the gradient — which makes recovery from a bad state faster.

There is also an algebraic bonus. When you stack softmax on top of logits and cross-entropy on top of softmax, a cancellation happens in the derivatives:

```
 d L(softmax(logits))
———————————————————— = probs - one_hot(target)
d logits
```

That is the entire gradient of softmax-plus-cross-entropy with respect to the raw logits. No exp, no log. Three characters of work in the code. Chapter 9 makes this explicit.

Averaging over the sequence

In MinAI, there are 8 output positions. We compute cross-entropy at each position and average:

```
L = (1/SEQ_LEN) * Σ_t (-log(probs[t][target[t]]))
```

The per-position factor of `1/SEQ_LEN` shows up in the gradient too, which is why in `backward()` you see the softmax-minus-one-hot divided by `SEQ_LEN`.

> **Read along:** `minai.cpp` Part 9, `compute_loss`. Four lines of arithmetic. The tiny `1e-12` floor on `p` is a safety net against the `log(0) = -inf` case; in practice it is never triggered during training, but it would blow up the first time a model got unlucky without it.

Chapter 9 — Calculus, the real deal: backpropagation

We have been dancing around this for eight chapters. Now we do it.

The situation

Our loss `L` is a scalar (a single number). It is computed from the parameters `(token_emb, pos_emb, Wq, Wk, Wv, W1, W2, Wout)` through a long chain of operations — matrix multiplies, softmax, residuals, ReLUs, and the cross-entropy. We want, for every parameter `θ`, the partial derivative `∂L/∂θ`. The full collection of these is the **gradient**, written `∇L`. Once we have it, gradient descent (Chapter 2) tells us the update: `θ := θ - α * ∂L/∂θ`.

The challenge: there are 2,240 such partial derivatives (or 175 billion, for GPT-3). Computing each one naively from scratch would be hopeless. Enter the **chain rule**.

The chain rule, reviewed

If `L = f(g(h(x)))`, then:

```
dL/dx = f'(g(h(x))) * g'(h(x)) * h'(x)
```

Each factor is evaluated at the correct intermediate value. Mechanically: "derivative of the whole = product of derivatives along the path from `x` to `L`".

For composed vector functions we get the same story with matrix multiplies instead of scalar multiplies:

```
dL/dx = dL/dy * dy/dx      (all matrices)
```

This is exactly what backpropagation is. It is the chain rule, applied layer by layer, from the output of the network backward to its parameters.

Two shapes of the chain rule you need

For a linear layer `Y = X @ W` (inputs `X`, weights `W`, output `Y`), given the gradient at the output `∂L/∂Y`, the gradient at the input and at the weights is:

```
∂L/∂X = ∂L/∂Y @ W^T
∂L/∂W = X^T @ ∂L/∂Y
```

That is a pair of matrix multiplies — the same basic arithmetic as the forward pass, just in a different shape. You can derive these by writing out one element of `Y` as a sum and applying the chain rule term by term. The result is so common in neural nets that it is worth memorizing like a phone number. You will see it four times in `backward_block` (for `Wout`, `Wq`, `Wk`, `Wv`) and twice more for the FFN (`W1`, `W2`).

The softmax + cross-entropy shortcut

Forward: `probs = softmax(logits)`, `L = -log(probs[target])`. If you grind through the chain rule using the explicit formulas for softmax and log, pages of algebra collapse to:

```
∂L/∂logits[i] = probs[i] - (1 if i == target else 0)
```

That is it. The gradient of cross-entropy through softmax is just `probs - one_hot(target)`. This is the algebraic miracle I have been teasing since Chapter 6. It is why every classifier pairs softmax with cross-entropy: the gradient computation has *no exp, no log*, just a subtraction.

A worked example: backward through one linear layer

Suppose `Y = X @ W`, with shapes:

- `X : [T, D]`
- `W : [D, V]`
- `Y : [T, V]`

Write one element of Y: `Y[t][v] = Σ_k X[t][k] * W[k][v]`. Partial derivative with respect to `W[a][b]`:

```
∂Y[t][v] / ∂W[a][b] = X[t][a]   if v == b
                      0         otherwise
```

Now apply chain rule: `∂L/∂W[a][b] = Σ_{t,v} ∂L/∂Y[t][v] * ∂Y[t][v]/∂W[a][b] = Σ_t X[t][a] * ∂L/∂Y[t][b]`.

Reassembled as a matrix: `∂L/∂W = X^T @ ∂L/∂Y`. Exactly the formula above.

The backward pass of the whole model

Run the forward pass, storing all intermediate activations. Then walk backward through the operations, applying the relevant chain-rule formula at each step. The intermediate gradients get reused — this is the efficient part of "backprop": instead of computing 2,240 derivatives from scratch, you compute each intermediate `∂L/∂activation` exactly *once* and reuse it.

There are only about six primitive operations whose local derivatives you need:

1. **Linear layer** `Y = X @ W`: the two formulas above.
2. **Addition** `Y = A + B`: gradient flows equally to both inputs (this is what makes residual connections so clean).
3. **Softmax** `y = softmax(x)`: `∂L/∂x_i = y_i * (∂L/∂y_i − Σ_j y_j * ∂L/∂y_j)`.
4. **ReLU** `y = max(0, x)`: gradient passes through where `x > 0`, zero elsewhere.
5. **Scalar multiply** `y = c*x`: gradient gets multiplied by `c` (for us, `c = 1/sqrt(D_MODEL)`).
6. **Lookup** `y = table[index]`: gradient accumulates in the looked-up slot.

That is the complete rule set. Every neural-network backward pass you will ever read is those six patterns composed.

> **Read along:** `minai.cpp` Part 10, `backward` and `backward_block`. Every step in those functions is one of the six rules above. Seriously, count — you can label each block of lines with one of the six numbers. This is the moment to sit with the code.

Chapter 10 — Stochastic gradient descent: training loop

With forward, loss, and backward in hand, the training loop is just Chapter 2's "four-step recipe" made into code:

```
for step in 1..NUM_STEPS:
    forward(tokens)                     # compute activations and probs
    loss = compute_loss(target)         # measure wrongness
    zero_param_grads()                  # wipe old gradients
    backward(tokens, target)            # compute ∂L/∂parameter for every parameter
    sgd_step()                          # parameter -= learning_rate * gradient
```

That is the entire loop.

Why it is “stochastic”

We call it SGD (Stochastic Gradient Descent) because in a real training setup, each step sees a *random* mini-batch of examples from a large dataset, rather than the full dataset. The randomness introduces noise into the gradient estimate, which — counterintuitively — helps the optimizer escape shallow local minima and find better parameter regions. In MinAI we only have one training example (the sequence `[0..7] → [7..0]`), so there is no randomness per step; our “SGD” is really just GD. But the code is identical.

Learning rate

The `α` in `θ := θ − α ∗ ∂L/∂θ` is the **learning rate**, and it is the single most important hyperparameter in all of machine learning. Too large: you overshoot the minimum and training oscillates or diverges. Too small: training takes forever. Different parameter groups often benefit from different rates, which is why `minai.cpp` defines `LR_EMB`, `LR_ATTN`, `LR_FFN`, `LR_OUT` separately.

Modern optimizers (Adam, AdamW, Lion) go further and automatically tune a per-parameter effective learning rate using running statistics of the gradients. They are strictly better for large-scale training. They also triple the memory usage, which was unaffordable on a PDP-11 — which is one reason MinAI sticks with hand-tuned fixed rates.

How do you know when it worked?

You watch the loss fall. In MinAI, run `./minai` and see the loss drop from ~2.28 to ~0.009 over 800 steps. Once loss is near zero and accuracy is 8/8, the training worked.

> **Read along:** `minai.cpp` Parts 11 (sgd_step) and 13 (train). Part 11 is three short loops; Part 13 is *the* training loop. If you have read this far, every line is understandable.

Chapter 11 — How calculus actually runs on silicon

We have spent a whole chapter on the calculus of backpropagation — gradients, chain rule, six primitive-operation rules. But a CPU has no *derivative* instruction. A GPU has no *gradient* instruction. They can add numbers, multiply numbers, load from memory, store to memory. That is all. So how does the abstract calculus from Chapter 9 actually become electricity moving through transistors?

Answer: we reduce calculus to arithmetic. Specifically, we express every derivative we need as a **matrix multiplication**, and then we run matrix multiplications on hardware built for exactly that.

Why it is always a matrix multiplication

Look back at the six-rule list from Chapter 9:

1. Linear layer, forward and backward: two matrix multiplies each.
2. Addition / residual connection: pointwise add.
3. Softmax: row-wise sum and divide.
4. ReLU: pointwise "zero if negative".
5. Scalar multiply: pointwise multiply by a constant.
6. Lookup: indexed read from a table.

Five of the six are trivial pointwise or row-wise operations. One — the linear layer — is a matrix multiplication. In a trained transformer, matrix multiplications account for the overwhelming majority of compute — typically 90% or more of both forward and backward pass time. Everything else is rounding error.

So the question "how do we run calculus on hardware?" reduces to "how do we run matrix multiplications fast?" And matrix multiplication is a question with a very concrete answer.

What a matrix multiply actually costs

Multiply two matrices `Y = X @ W`. Let X have shape `[M, K]` and W have shape `[K, N]`. The output Y has shape `[M, N]`, and each of its elements is a dot product of length K — that is, K multiply-adds. There are `M × N` output elements. Total arithmetic work:

```
M * N * K  multiply-adds
```

A multiply-add is the simplest useful operation a computer can do. Grab two numbers, multiply them, add to a running total. On modern hardware this is a single instruction called an **FMA** (fused multiply-add). Every computer — CPU or GPU — is essentially an FMA machine with extra life support.

To make this concrete: one attention layer in GPT-3 computes a Q @ K^T that is roughly `2048 × 12288 × 12288 ≈ 300 billion FMAs`. Per forward pass, per layer. The backward pass doubles it. Across all 96 layers and trillions of training tokens, the total compute to train GPT-4 has been estimated at roughly `10^25` FMAs. Ten septillion. The hardware exists to make those FMAs finish in a reasonable number of months.

Tensors: just "arrays with more than two dimensions"

You will hear the word *tensor* everywhere in modern ML (PyTorch's core type, "TensorFlow", Google's "Tensor Processing Units"). In this context, a tensor is nothing more than a multi-dimensional array of numbers. A vector is a 1-tensor. A matrix is a 2-tensor. The 3-tensor `X_block[SEQ_LEN][D_MODEL]` (stacked across `MAX_BLOCKS+1`, making it 3D total) is exactly what MinAI stores, and exactly what PyTorch would call a tensor.

Operations on tensors are generalizations of operations on matrices. A "batched matrix multiply" is a pile of matrix multiplies in parallel — once you can do one quickly, you can do a batch quickly. The word *tensor* in software doesn't carry the physics meaning; it's just "a multi-dimensional array with nice operations."

Reducing calculus to arithmetic, in practice

Here is the practical flow every framework (and MinAI) follows:

1. **The user writes a forward pass** — a specific composition of the six-or-so primitive operations (linear layer, softmax, ReLU, add, etc.).
2. **Each primitive has a hand-derived backward formula**. Somebody did the calculus once. For the linear layer, it is the `Xᵀ @ dY` and `dY @ Wᵀ` pair. For ReLU, it is "pass through where input was positive, zero otherwise". These formulas live in a library and are reused forever.
3. **The backward pass is the forward pass run in reverse**, applying the pre-derived formulas at each step, with matrix multiplies where the forward had matrix multiplies.

The result: *no calculus is ever done by the computer at runtime.* The calculus was done by a human, once, symbolically, to produce formulas. What runs on the silicon is just repeated matrix multiplications. This is the critical insight the user rarely sees stated plainly.

Modern frameworks add an automation layer: **automatic differentiation** (autograd). The user writes only the forward pass; the library records every primitive op called, then walks the list in reverse applying the library's stored backward formulas. The computer is still not "doing calculus" — it is still just dispatching matrix multiplies. The autograd layer just figures out *which* matrix multiplies to dispatch, so the human doesn't have to.

MinAI does not have autograd. The backward pass in `minai.cpp` Part 10 is hand-coded, step by step, each block labeled with which rule applies. Read it once and you have seen precisely what a framework like PyTorch is doing behind a cleaner API.

What CPUs and GPUs can do per second

A modern CPU core, using SIMD instructions (NEON on Apple Silicon, AVX on x86), can do roughly 16–32 FMAs per cycle. With a 3 GHz clock and 8–16 cores, a good CPU tops out at a few hundred billion FMAs per second. Sounds huge. Now divide 10^25 training FMAs for GPT-4 by a few hundred billion per second. That is a few thousand years on a single CPU.

You need something much better. Enter the GPU.

> **Read along:** `minai.cpp` Part 8 — every forward-pass matrix multiplication has the identical triple-nested loop structure:
>
> ```
> for (int t = 0; t < ROWS; ++t)
>  for (int j = 0; j < COLS; ++j) {
>  float s = 0;
>  for (int i = 0; i < INNER; ++i) s += A[t][i] * B[i][j];
>  Y[t][j] = s;
>  }
> ```

> That innermost line — `s += A[t][i] * B[i][j]` — is one FMA. Every matrix multiply, anywhere in any AI system in the world, is a way to dispatch billions or trillions of those FMAs to as many cores as possible at once.

Chapter 12 — GPUs and memory: why hardware is the limit right now

This is the chapter that explains why NVIDIA is suddenly one of the most valuable companies in the world.

CPU vs GPU, in one paragraph

A CPU has a handful of very clever cores. Each one can run arbitrary branchy code — web servers, compilers, string parsing, game logic. Deep caches, out-of-order execution, branch prediction, speculative loads. A CPU is a generalist.

A GPU has thousands of much simpler cores. Each core is bad at branching and arbitrary code. They are built to do *the same arithmetic operation on a huge pile of numbers at once*. That shape — "same op, lots of data" — is exactly matrix multiplication. An NVIDIA H100 has about 14,000 general shader cores *plus* a few hundred specialized **Tensor Cores**, each of which performs a small matrix multiplication (e.g., 4×4 × 4×4) in a single instruction. Peak throughput at low precision: around 2,000 trillion (two quadrillion) FMAs per second — roughly 6,000× a strong CPU. That ratio is the entire reason AI training happens on GPUs.

GPUs were not invented for AI. They were invented for 3D graphics, which happens to also be "apply the same matrix op to a pile of vertices/pixels". It is an accident of history that graphics cards from the 1990s became the computation backbone of the 2020s AI boom. (Except it isn't entirely an accident: dense matrix math is fundamental to a lot of physics and engineering.)

The memory wall — why compute is no longer the bottleneck

Here is the part almost nobody outside the field knows. The bottleneck of modern AI is not "how fast can we multiply". It is "how fast can we **move** numbers from memory to the arithmetic units". This is called the **memory wall**, and it has dominated hardware design for over a decade.

The arithmetic is fast. You load W and X into registers, execute an FMA, done in a nanosecond. But *loading* W and X from DRAM into registers takes *hundreds* of nanoseconds. If you have to load fresh data for every FMA, the multiplier sits idle 99% of the time. The whole game of modern GPU design is to keep the multipliers fed.

Three numbers worth keeping in your head:

1. **How big are the models?** GPT-3 has 175 billion parameters. At 2 bytes each (16-bit floats), that is 350 gigabytes *just for the weights*. A single NVIDIA H100 GPU has 80 GB of on-chip memory (a special kind called **HBM**, for High Bandwidth Memory). You cannot fit GPT-3 on one H100. You need many GPUs

whose memory is pooled via high-speed interconnects (NVLink, InfiniBand). A GPT-4-scale training run might use thousands of GPUs simultaneously, coordinated.

2. **Activation memory during training.** Backprop needs every intermediate activation from the forward pass. For large models this rivals parameter memory. Techniques like **gradient checkpointing** (throw away activations on the forward pass and recompute them during backward) explicitly trade compute for memory — because memory is scarcer.

3. **Bandwidth matters more than capacity.** Your laptop's DDR5 system RAM moves about 50 GB/s. An H100's HBM moves about 3,000 GB/s — 60× faster. That is why model weights live on the GPU, not in system RAM: the CPU-to-GPU link (PCIe) is too slow to feed the multipliers. *Keeping weights close to the compute is the whole point.*

Inference has a memory problem too: the KV cache

Even after a model is trained, running it costs memory. When you ask ChatGPT a question, the model generates one token at a time. For each new token, attention needs access to the K and V vectors of every previous token (see Chapter 5). If it re-derived them from scratch every time, it would re-process the entire conversation every single token. Unacceptably slow.

The fix: cache K and V for every seen token in GPU memory. This is called the **KV cache**. Its size grows linearly with the length of the conversation. For GPT-4 generating a 100,000-token response, the KV cache alone is tens of gigabytes.

This is why long context windows are expensive and why context length is a hardware story, not an algorithmic one. Longer context = more RAM needed on the GPU. Full stop.

Why "right now"

Three things collided around 2020–2023 to create the current moment:

1. **Scaling laws held.** For years, researchers dreaded diminishing returns: double the parameters, get nothing. Instead, doubling kept working, and kept working, and kept working. The "bitter lesson" of AI is that the thing you'd most want *not* to be true turned out to be true: brute force wins.

2. **Transformers generalized.** The same architecture used for language turned out to work for images (Vision Transformers), audio, protein structures, robot control. One hardware stack (GPUs with HBM) now serves almost every large AI workload.

3. **Hardware barely kept up.** Training GPT-4-scale models requires enough HBM, enough interconnect bandwidth, and enough power delivery that only a handful of companies in the world can assemble the necessary cluster. HBM is the choke point: HBM manufacturers (SK Hynix, Samsung, Micron) sell everything they can produce for the next several years. The latest NVIDIA generation, Blackwell, did not dramatically speed up the multipliers compared to the previous generation; what it *did* was double or triple the amount of memory per GPU.

The headline version: **compute grew faster than memory bandwidth for twenty years. Now AI has reached the scale where memory bandwidth is the limit.** Every engineering effort in AI hardware right now is about moving bytes into the multipliers faster. That is why companies spend billions on GPU clusters, why the stock market cares so deeply about a company whose main product is a *memory-saturating math engine*, and why headlines about "datacenter power consumption" and "more GPUs" and "more HBM" feel omnipresent. That is the curtain.

> **Read along:** MinAI has none of this. Our model is 2,240 floats, 9 KB of memory, trains in under a second on any CPU. Every concept is present, but the hardware reality is absent — because a model this small doesn't need a GPU, doesn't need HBM, doesn't have a KV cache worth caching. Chapter 11's forward-pass triple-nested loop *would run exactly the same pattern* on a GPU, just ten million times faster because the GPU dispatches ten thousand FMAs in parallel instead of sixteen.

Chapter 13 — From MinAI to GPT-4

You now understand every mechanism in every transformer. The scale-up to a real LLM is purely *quantitative*. Here is the full list of differences:

MinAI (with defaults)	GPT-4 (estimated)
1 transformer block	~96 blocks stacked
1 attention head per block	~96 attention heads per block, each a mini version of our single head, concatenated
`D_MODEL = 16`	`D_MODEL ≈ 12,288` or more
`D_FF = 32` (2× `D_MODEL`)	`D_FF = 4 × D_MODEL`
`VOCAB = 10` (digits)	`VOCAB ≈ 100,000` (subword pieces)
`SEQ_LEN = 8`	`SEQ_LEN` in the hundreds of thousands
optional LayerNorm (`--layernorm=1`)	always-on LayerNorm before each sub-block for training stability
optional causal mask (`--causal=1`)	always-on causal mask (for generative models)
one fixed training example	trillions of tokens of text
~2,240 parameters	~1,000,000,000,000+ parameters

Multi-head attention is worth a paragraph. Instead of one set of `(Wq, Wk, Wv)` of full size `[D_MODEL × D_MODEL]`, you have `H` sets of `[D_MODEL × (D_MODEL/H)]`. Each head performs the Chapter-5 attention independently on its share of the dimensions. Then the `H` outputs are concatenated back into a `D_MODEL` vector. Why? Different heads learn to attend for different reasons — one head might focus on syntactic agreement, another on topical relevance. You get `H` distinct attention patterns instead of one.

LayerNorm is a normalization applied to each token's vector before each sub-block: `LayerNorm(x) = (x - mean(x)) / std(x) * gain + bias`. It keeps activations from growing or shrinking as depth grows, which makes 100-layer stacks trainable. MinAI has this now (as of the `--layernorm=1` flag) — see the dedicated section at the end of Chapter 7 for the math and why it works, or just run `./minai --blocks=32 --layernorm=0` to watch gradients explode into `NaN` and then `./minai --blocks=32 --layernorm=1` to watch the same depth train happily.

None of this changes the ideas in Chapters 1–10. Multi-head attention is attention, four times over, in parallel. The depth is the same loop. The training is the same SGD. The loss is the same cross-entropy.

If this is all one big program, then the difference between MinAI and GPT-4 is that GPT-4 is a *much bigger* one big program. The bitter lesson of the last decade is that, at least for language, "more of the same" scales shockingly well. Nobody predicted how far it would go.

Chapter 14 — A guided tour: grow your understanding through the flags

You've read twelve chapters of theory and one chapter scaling it up. This chapter is hands-on. Run these commands in order, and after each one look at two things: the loss curve printed at the end, and the attention matrix from the demo. Each command changes *one idea* at a time and lets you see the effect.

`minai.cpp` prints a loss sparkline like this after every run:

```
Loss curve (800 steps, 60 columns, higher bars = higher loss):
  2.280 |▅▅▅
```

This is the starting point. One transformer layer (attention + FFN), fixed input `[0..7]`, task is “reverse”. Watch the loss drop to about 0.008 and all 8 positions become correct. This is where everything in Chapters 1–10 is visible in action. Open `minai.cpp` Part 8 (`forward_block`) and Part 10 (`backward_block`) alongside; the training log that fills your terminal is literally that code being called 800 times.

What to notice. The first 20 steps are shown one-by-one so you can see the rapid early descent from loss 2.28 to 2.21 happens in the first *twenty* steps alone. That is gradient descent biting. Look at the attention matrix — probably it is surprisingly uniform. The model is NOT reversing by attention; it is using the output projection to memorize the mapping. That’s a first lesson in “the model finds a shortcut if the task lets it.”

Step 2 — More layers, same task

```
./minai --blocks=2
```

Stack two transformer layers. Parameters double (4,032 from 2,240). Final loss should go an order of magnitude lower (around 0.002). Look at block 1’s attention matrix — you’ll start to see peaks on specific input positions (especially rows 6 and 7), suggesting the second layer is beginning to learn actual positional reversal.

The lesson. More capacity = lower loss, but also different *internal strategies*. A 2-layer model often finds a cleaner solution than a 1-layer model squeezed to do everything.

Step 3 — Force generalization with random inputs

```
./minai --random=1 --steps=5000
```

Now the model sees a fresh random 8-digit sequence every training step. It cannot memorize; it must learn the *rule*. The training log gains a held-out accuracy column — performance on 64 sequences the model has never seen. Watch the train-accuracy column jitter wildly (0% / 12.5% / 25% / 0% / ...) — that is the single-example noise in action. Held-out accuracy crawls upward, maybe to 15-25% by step 5000. The model is struggling, and with batch size 1 it may not grok at all within patience. Step 5 will show why bigger batches change this.

The lesson. This is the moment the word “generalization” becomes concrete. Memorizing is easy; learning a rule is hard. Real LLMs are trained almost entirely in this regime — every example seen once, with held-out data measuring whether the model is actually *learning* vs *storing*. The gap between train accuracy (which jitters wildly from step to step) and held-out accuracy (which rises smoothly) is the heartbeat of machine learning.

Step 4 — Causal masking actually bites, and here is exactly why

```
./minai --random=1 --causal=1 --steps=2000
```

Same setup as before, but attention can only look backward. For reversal, position 0 needs to see position 7 — and now can't. Over 2000 steps you'll see held-out accuracy stuck at about 10–15%, barely above the random baseline. You would be forgiven for concluding the model just "can't learn" this task. But if you run it much longer:

```
./minai --random=1 --causal=1 --steps=200000
```

Something fascinating happens. For roughly the first 5,500 steps the loss stays flat around 2.30 and held-out accuracy is stuck at ~10%. Then, suddenly, between step 5,500 and step 9,000, the loss plunges from 2.30 to 1.30 and held-out accuracy leaps from 17% to 55%. This sudden breakthrough after a long apparent stall is a real named phenomenon, **grokking** (Power et al., 2022). The model spends thousands of steps quietly reorganizing its internal representation without visible progress, then crosses a threshold and the loss collapses almost instantly. From that point on, held-out plateaus at about **55%**, and *will never go higher no matter how long you train*.

Why exactly 55%? A counting argument.

With causal masking on an 8-digit reversal, each output position `t` needs to predict `input[7-t]` while only being allowed to see `input[0..t]`:

output `t`	needs `input[7-t]`	allowed to see	best possible
0	`input[7]`	`input[0]`	10% (random guess)
1	`input[6]`	`input[0..1]`	10%
2	`input[5]`	`input[0..2]`	10%
3	`input[4]`	`input[0..3]`	10%
4	`input[3]`	`input[0..4]` ← includes `input[3]`	100%
5	`input[2]`	`input[0..5]` ← includes `input[2]`	100%
6	`input[1]`	`input[0..6]` ← includes `input[1]`	100%
7	`input[0]`	`input[0..7]` ← includes `input[0]`	100%

The last four positions *can* see the digit they need to retrieve and learn to reproduce it perfectly. The first four positions are being asked to predict an input they are not allowed to see. Averaging across all eight positions:

```
(4 × 10% + 4 × 100%) / 8 = 55%
```

That is the architectural ceiling. Your model converges to 55.5%. That is not a coincidence — it is the ceiling being hit.

Why the ceiling is not just "hard" — it is mathematically impossible to beat

In `--random=1` mode, each input digit is drawn **independently and uniformly** from 0-9. So for output position 0, which may only observe `input[0]` but needs to predict `input[7]`:

```
P(input[7] | input[0]) = P(input[7])
```

The conditional probability equals the unconditional probability. Knowing `input[0]` gives you **zero bits of information** about `input[7]` — they are as unrelated as two separate dice rolls. This is a property of the *data generator*, not the *model*. No function of `input[0]` can predict `input[7]` better than the marginal distribution, because no such function exists. Not a bigger model, not more training, not a cleverer architecture. If the information the model would need is not in its input, no model can conjure it.

The four unreachable positions therefore converge to cross-entropy loss `log(10) ≈ 2.30` (uniform guessing). The four reachable positions converge to essentially zero loss. Averaged over all eight positions:

```
loss ≈ (4 × 2.30 + 4 × 0) / 8 ≈ 1.15
```

Which is exactly the plateau your program settles at after grokking. The ceiling is not a flaw in the model or a failure of training — it is the model being *correct*: doing precisely what the available information allows, no more, no less.

But real language models use causal masks and work fine — why?

Because natural language is loaded with statistical structure. "The cat sat on the ___" strongly predicts "mat" or "couch" or "floor". The conditional distribution `P(next_word | previous_words)` is *vastly* narrower than the unconditional `P(any_word)`. Past tokens carry enormous information about future tokens, so a causal language model can extract meaningful signal from what it is allowed to see.

Random digit reversal has the *opposite* property: past positions carry no information about future positions, because the data was generated independently. Causal masking is *fine* for tasks where the future depends on the past; it is *fatal* for tasks where the future is statistically independent of the past.

The takeaway — a core ML engineering skill

When a model's accuracy plateaus, there are two very different possibilities:

- **(a)** It has not found the solution yet. More compute, more data, better architecture, different hyperparameters might unstick it.
- **(b)** The information the model would need to improve is not in its input. No amount of anything will help.

Distinguishing these two cases is one of the core skills of a machine-learning engineer. Here — because we know exactly how the data is generated — we can *prove* this is case (b). The model learned every bit there was to learn; the remainder is not there to be learned. That proof comes out of the table above plus the independence property of the data. It is the same style of argument Claude Shannon formalized in 1948 for information theory, and it still runs every decision in the field.

Step 5 — Batch training smooths the curve AND unlocks grokking

```
./minai --random=1 --batch=16 --steps=15000
```

Each training step now averages gradients over 16 random sequences instead of one. Compare the loss sparkline to Step 3's: this one is visibly smoother, and critically the model now has enough clean signal to actually learn the rule. Expect held-out accuracy to stay ~10% for several thousand steps, then **grok** — a sudden breakthrough where held-out accuracy climbs from ~20% to ~100% over a window of 2–3 thousand steps. The exact step count of the breakthrough varies with the RNG seed, but it almost always happens well before step 15000.

The lesson. Every real LLM trains with large batches — GPT-3 used batches of millions of tokens per step. The effect is exactly what you see here: the gradient direction is less noisy, so the optimizer can take more confident steps. This is also why GPUs are useful: a batched matrix multiply of 16 sequences fills a GPU's parallel arithmetic units much better than one sequence does (see Chapter 12).

Step 6 — Sequence length = quadratic attention cost

```
./minai --seq_len=16 --steps=2000
```

Double the sequence length from 8 to 16. Watch the per-step log; it will run visibly slower. Attention's score matrix is now 16×16 = 256 entries per block instead of 8×8 = 64. That `O(N²)` scaling is the reason long-context LLMs are expensive.

The lesson. Every scaling number you read in the press — "GPT-4 supports 128k tokens" — translates to "the inner attention operation costs $128{,}000^2$ = 16 billion entries per layer". That is the memory and compute price of long context, directly visible in our program slowing down.

Step 7 — A harder task needs more capacity

```
./minai --task=sort --blocks=2 --random=1 --batch=16 --layernorm=1 --steps=10000
```

Switch the task to "sort the input ascending". This is harder than reversal: each output position needs to know the *entire* input to decide its rank. Run it with two blocks, batch 16, and LayerNorm on for stability, and give it

10k steps so grokking has time to land. Watch held-out accuracy climb substantially above 10% even though sort is much harder than reverse.

The lesson. Architecture has to match task difficulty. Try `--task=sort --blocks=1 --ffn=0` — you'll likely see it fail to learn at all, because "half a layer" doesn't have the computational depth to do sorting. Bigger models aren't always better on easy tasks (too much capacity overfits), but they become *necessary* on hard ones. This is the scaling law in miniature.

Step 8 — The local task

```
./minai --task=mod_sum --random=1 --batch=16 --layernorm=1 --steps=3000
```

Target[t] = (input[t] + input[t+1]) mod 10. Each output depends on only two adjacent inputs. Attention barely has to do anything — the FFN does most of the work. Accuracy should climb toward 100% within the 3000-step budget. Try it again with `--ffn=0` and watch it struggle — FFN is doing the arithmetic here.

The lesson. Different tasks stress different parts of the transformer. Attention = cross-position communication; FFN = per-position computation. A truly general model has both because different sub-problems need each.

Step 9 — Deep stacks explode without LayerNorm

```
./minai --blocks=32 --layernorm=0 --steps=200
```

Thirty-two layers, no normalization. Watch what happens — within about 150 steps the loss column prints `nan` and accuracy locks at 1/8. The 32-layer model's gradients have exploded to infinity and then past it. The model is dead. This is not a bug in MinAI; it is exactly what happens to any deep neural network without normalization. It is the wall that stopped transformers from going deep before LayerNorm (and its cousin BatchNorm) solved the problem.

The lesson. Architecture has numerical constraints that compound with depth. "Just stack more layers" doesn't work by itself; you need machinery to keep the scales of activations and gradients under control across all those layers.

Step 10 — LayerNorm makes the same depth trainable

```
./minai --blocks=32 --layernorm=1 --steps=1000
```

Exact same architecture, same depth, same training signal. Only change: LayerNorm added. Loss drops smoothly, accuracy hits 8/8 long before step 1000 and then just keeps driving the loss down. Same problem, entirely different outcome, because of one normalization step per sub-layer.

The lesson. LayerNorm is the single most important piece of machinery separating 2017-era shallow transformers from the 100-layer models that followed. Once you have it, depth stops being limited by numerics and starts being limited by compute.

Step 11 — GPT-3 depth, in a single-file C++ toy

```
./minai --blocks=96 --layernorm=1 --steps=2000
```

Ninety-six layers. The exact layer count of GPT-3. In ~820 lines of C++ on a laptop. It will run for a few minutes because 96 layers × 2000 steps × many small matrix multiplies, and it converges slowly because we still only have `D_MODEL=16` width to spread 96 layers across. But it *trains*. No crashes, no NaN, loss falling steadily, accuracy heading to 8/8. That is the skeleton of GPT-3, running in full, on your own machine.

The lesson. This is the moment MinAI stops being a toy demo and starts being a true miniature of a real LLM. Every idea you read about in frontier-model papers — depth, residuals, pre-LN, batch training, attention, softmax, cross-entropy — is present here. The only difference is magnitude. Same dials, same physics.

Step 12 — Hierarchical weight quantization (the "organist" trick)

```
./minai --random=1 --batch=16 --blocks=4 --layernorm=1 --steps=3000 --extra_demos=1
```

After training finishes, the program runs three bonus demos. The first quantizes the trained `Wout` matrix three ways — Q8 flat, Q4 flat, Q4 with hierarchical scaling — and reports the precision cost of each. The hierarchical Q4 uses a "feet" (global) scale, a "left hand" (per-column) scale, and a "right hand" (per-weight 4-bit) index. This is the compression scheme behind llama.cpp's Q4_K_M format that lets Llama-2 70B run on a 64 GB laptop.

The lesson. You don't have to store every weight at 32 bits. With a hierarchy of scales you can get down to ~5 effective bits per weight while keeping most of the precision. Bandwidth is the bottleneck (Chapter 12); fitting 4-6× more weights per byte loaded is 4-6× more tokens produced. This is how 2025-era open-weight LLMs reach consumer hardware.

Step 13 — Speculative decoding (fast cortex + slow cortex)

Same command as Step 12. The second demo trains a smaller "draft" model (half the block count of the big one, everything else the same), then measures how often the draft and big model agree on the held-out set. The agreement rate is exactly the *expected acceptance rate* of speculative decoding — the mainstream inference optimization used by every frontier model (GPT-4, Claude, Gemini, llama.cpp, vLLM).

The lesson. The big model's forward pass is dominated by loading its weights from memory, not by computing anything. Loading those weights once to verify `K=4` draft-proposed tokens costs almost the same as loading them once to produce `1` token. So if the draft is right 80% of the time, the big model effectively

produces ~3 tokens per forward pass — a near-3× throughput win at zero quality cost because the big model has final say. It maps almost exactly onto the fast-subcortical vs slow-cortical split in biological brains.

Step 14 — KV cache tiering (Miller's 7 in silicon)

Same command as Step 12. The third demo sweeps a "hot working memory" size `W` from 0 to `seq_len`, re-running the forward pass with `K` / `V` quantized to Q8 for every position older than `W`. It prints accuracy at each `W`, with Miller's 7 marked.

The lesson. Human working memory holds about 7±2 items; older memories get compressed into longer-term storage that is slower but much larger. Production LLMs do *exactly* this with their KV cache: keep the most recent handful of tokens at full precision, progressively quantize older ones, evict the oldest. This is how 1M-token-context models become tractable — no GPU has a terabyte of HBM, so the cache has to be tiered. On MinAI specifically the Q8 round-trip is too fine-grained to move the accuracy needle, as the note at the end of the demo explains. It is also the only extension in this program that came *directly* from a human-brain observation: you already know the engineering answer because your skull is solving the same problem.

Putting it together

By now you should have:

- Seen gradient descent work in real time (Step 1).
- Felt the difference between memorization and generalization (Step 3).
- Seen an architectural choice break a model (Step 4).
- Watched batching smooth training noise (Step 5).
- Felt `O(N²)` attention slow down (Step 6).
- Matched architecture capacity to task difficulty (Step 7).
- Seen that FFN matters on some tasks and not others (Step 8).
- Watched a deep network explode into NaN without normalization (Step 9).
- Watched the same depth train cleanly with it (Step 10).
- Actually trained a 96-layer transformer on your own machine (Step 11).
- Compressed a weight matrix with a three-level scale hierarchy (Step 12).
- Measured the draft-model agreement rate that drives speculative decoding (Step 13).
- Quantized old tokens and seen the biological-working-memory / LLM-KV-cache alignment (Step 14).

That is not a metaphor for what it's like to train an LLM. That *is* what training an LLM is. The only thing GPT-4's researchers are doing differently is doing it with a million times more of everything — more parameters, more data, more compute — while fighting the same fundamental tradeoffs between capacity and overfitting, speed and batch size, context length and memory. Same dials, same curves, same lessons.

The curtain is down.

Appendix A — The PDP-11 fixed-point story

`minai.cpp` descends from a program Damien Boureille wrote in PDP-11 assembly for a 1975 minicomputer. The PDP-11 has no floating-point unit in its base configuration. How do you train a neural network — full of multiplications and softmaxes — when you only have integer arithmetic?

Q8.8 fixed-point

Represent a real number `r` by storing `round(r * 256)` as a 16-bit integer. The low 8 bits now hold the fractional part. `1.0` is stored as `256`. `0.5` is stored as `128`. Add two Q8 numbers: just add the integers. Multiply two Q8 numbers: their integer product is in Q16 (both 8-bit fractions merged, 16 fractional bits total), so shift right by 8 to get back to Q8. One instruction. The PDP-11's `MUL` happens to put its 32-bit product in a register pair perfectly sized for this.

This is an **integer** encoding of real numbers. It has a range of about `[-128, 128)` and a precision of about 1/256. Good enough for the forward pass of a small network.

For gradients, which are smaller numbers, you use **Q15** (15 fractional bits instead of 8). Precision 1/32768. And for long-running accumulators (like weight update sums) you use Q16 in a 32-bit integer. The three formats are chosen so that pairs of them multiply cleanly with the PDP-11's instruction set.

Softmax without exp

The PDP-11 has no `exp` function. The Boureille code precomputes a table of 256 values: `exp(-k/8)` for `k = 0..255`, each stored as a Q8 integer. To compute softmax:

1. Find the max Q8 value.
2. For each `x_i`, compute `(max - x_i) >> 5`. This gives an integer in `[0, 255]` or beyond.
3. Look up `exp(-idx/8)` from the table (or return 0 if the index is too large).
4. Divide by the sum.

No floating-point required at any step. The `>> 5` shift is the magic: converting a Q8 difference to an integer table index equal to `8 * (max - x_i)` in real-value terms — which is the `k` for `exp(-k/8)`. This is the softmax that runs in the real PDP-11 assembly. You can run a demo of it at the end of the C++ program: see Part 15 of `minai.cpp`.

This is still the future

"Quantization" — representing weights and activations with fewer bits — is an active area of modern LLM research. Int8, int4, even int2 quantizations are used to shrink multi-billion-parameter models into formats that can run on a phone. The underlying trick is identical to Boureille's: store real numbers as scaled integers, do arithmetic in integers, convert back only when needed. The PDP-11 was doing it in 1975 because it had no FPU. Apple's Neural Engine does it in 2025 because it's faster and uses less power. Same trick, fifty years apart.

Appendix B — Exercises

These are optional but deepen understanding.

1. **Count parameters.** Before running the program, calculate by hand the parameter count for `--blocks=3 --ffn=1`. Then run it and check. (Answer in the Part 1 comment of `minai.cpp`.)
2. **Break it.** Comment out the line that adds `pos_emb` in Part 8. Recompile. Retrain. What happens and why?
3. **Attention observation.** Run `./minai --blocks=2`. Look at the attention matrix of block 1. Which row has the highest peak on column 0? Why is this the "right" behavior for a reversal task?
4. **Causal failure.** Run `./minai --causal=1`. The accuracy goes to 8/8 anyway. Explain why, referring to what the fixed input lets the model do.
5. **Gradient by hand.** Take the three-variable function `L(a, b, c) = (a * b - c)^2`. Compute the three partial derivatives `∂L/∂a`, `∂L/∂b`, `∂L/∂c` by hand. Identify which of the "six rules" from Chapter 9 each one uses.
6. **Softmax translation invariance.** Pick any three numbers `x_0, x_1, x_2`. Compute `softmax` on them. Add 100 to all three. Compute again. Confirm they are identical up to rounding.
7. **Q8 sanity check.** In Q8.8, what integer represents `0.125`? What integer represents `-1.5`? What happens if you multiply the first by itself and shift right by 8 — do you get the Q8 representation of `0.015625`?
8. **Read a paper.** The original attention paper is "Attention Is All You Need" (Vaswani et al., 2017). With Chapters 1–13 in hand, you have the vocabulary to read it end to end. Try. (Skip the BLEU score part if you don't care; the rest is in your reach.)

End.

If this book helped, pass it along. If something in it was wrong or unclear, the source code is in the same directory — go check, go correct.